Jordan Glass

Hoboken, New Jersey 07030
jordanglass421@gmail.com | linkedin.com/in/jordan-glass-one | github.com/jordanglass21

Summary

Software engineer with production experience building and testing Web APIs in Node.js/NestJS, TypeScript, and PostgreSQL, and project experience with Go, Python, Java, C#, and AWS infrastructure. Brings four years running operations for an $8M/yr retail business, translating business needs into technical solutions and working directly with vendors, staff, and customers.

Education

Northeastern University, Khoury College of Computer Sciences Aug. 2026
Master of Science in Computer Science *Boston, MA*
Relevant coursework: Database Management Systems, Building Scalable Distributed Systems

Binghamton University – SUNY May 2019
B.A. Geography Computer Applications, Minor in CS *Binghamton, NY*

Technical Skills

- **Languages:** TypeScript, JavaScript, Python, SQL, Go, Java, C#, Bash, HTML/CSS
- **Databases:** PostgreSQL, MySQL, DynamoDB, SQLite
- **Cloud & DevOps:** AWS (ECS, S3, CloudWatch), Terraform, Docker, Linux, Git, GitHub Actions, CI/CD
- **Frameworks, APIs & Testing:** Node.js, NestJS, Express, Next.js, React, .NET, Spring Boot, REST, OpenAPI, gRPC, Jest, k6

Experience

Software Engineering Co-op Sep. 2025 – Dec. 2025
Hydrow *Boston, MA*

- Designed and built progress summary and calendar REST APIs in NestJS, TypeScript, and PostgreSQL, generating dynamic stats and milestones across multiple time ranges and workout types, with streak tracking.
- Wrote OpenAPI specs for five subscription authorization APIs as a contract for mobile and web teams, then built them to let owners grant, revoke, and bulk-manage permissions with safeguards against accidental admin removal.
- Traced and fixed user-facing data bugs, including a weight rounding mismatch between live workouts and personal records and strength users being shown rowing content.
- Wrote Jest unit and end-to-end tests for every feature shipped, releasing weekly through CI/CD across Dockerized services with code reviews and sprint demos.

Operations Manager Feb. 2020 – Aug. 2024
Star Wines and Liquors *Monroe, NY*

- Integrated third-party platforms (DoorDash, Vivino) and launched digital campaigns for an $8M/yr retail business, generating over $800K in annual online revenue.
- Managed a team of 20 and served as the main contact for vendors, distributors, and customers, negotiating pricing, delivery, and contract terms.

Projects

NEUDFS: Cloud-Native Distributed File System | Team of 4 | *Go, gRPC, AWS (S3, DynamoDB), Terraform, Docker, k6*

- Co-built a distributed file system for university courses: a Go gRPC server behind an AWS Network Load Balancer that scales by adding servers, with file data in S3 and metadata and role-based permissions in DynamoDB.
- Owned the S3 infrastructure in Terraform, file and folder rename and delete (finding and fixing a race condition in delete), and permission checks for creating folders.
- Load-tested with 19 k6 scenarios, including concurrent renames, upload races, 50 MB file streams, and a 10-minute soak test; none of six kinds of attempted unauthorized access got through.

ASL Drill: Fingerspelling Trainer | *TypeScript, Next.js, MediaPipe, Python, scikit-learn, Jest* asldrill.jordanglass.dev

- Built a webcam trainer that grades each fingerspelled letter in real time, with MediaPipe hand tracking running entirely in the browser so video never leaves the device.
- Trained a small neural network in Python with scikit-learn, exported its weights as JSON, and ran inference in plain TypeScript with no ML runtime; shared fixtures keep both pipelines in exact agreement.
- Detected motion letters (J, Z) and double letters with geometric checks on a one-second landmark buffer; wrote letter confirmation as a pure, unit-tested state machine, with CI on every pull request.

AssetSync: Legacy Data Reconciliation Tool | *C#, .NET 10, Entity Framework Core, SQLite, Java, Spring Boot, React*

- Built a tool that audits a modern inventory database against a legacy system, with a C# .NET 10 API, a Java Spring Boot service simulating the legacy database, and a React dashboard communicating over HTTP/JSON.
- Matched about 300,000 rows of real public sales data in constant time per row by indexing legacy records by item, year, and month, using a one-cent tolerance so floating-point noise isn't reported as drift.
- Wrote an end-to-end test that injects drift through the legacy service, verifies the audit catches it, and resets.